

Adding basic function

Hi christiaen and felix.

i hope this assignment will impress you guys, and i am really sorry of what happened in the final assesment.

in this extra assignment, i will add some basics instruction. starting from R type

R-Type

XOR

how does XOR works in the first place

XOR, so both of the number is converted to binary

and then they see both of them, if two of it is the same, then result 0, if two is different, then result will be one

its like this

15 = 1111

10 = 1010

sooo XOR, would be 0101 which is 5

and according to the datasheet, this is the opcode, func7, func3.

0110011 (51)	100	0000000	R	xor	rd, rs1, rs2	xor	rd = rs1 ^ rs2
--------------	-----	---------	---	-----	--------------	-----	----------------

here, i will edit my control unit to have XOR, which in the func3 be 100

```
1 if funct7 = "0000000" then
  if funct3 = "000" then
    ALUControl <= "010"; -- ADD [7, 8]
  elif funct3 = "100" then
    ALUControl <= "011"; -- XOR
  elif funct3 = "111" then
    ALUControl <= "000"; -- AND [7, 8]
  else
    ALUControl <= "111"; -- Invalid
```

okay, after this, it will give alu 011, which i set in my alu to do XOR

```
begin
  case ALUControl is
    when "000" => -- AND
      result := UNSIGNED(A and B);
    when "001" => -- OR
      result := UNSIGNED(A or B);
    when "010" => -- ADD
      result := UNSIGNED(A) + UNSIGNED(B);
    when "110" => -- SUB
      result := UNSIGNED(A) - UNSIGNED(B);
    when "111" => -- SLT
      if UNSIGNED(A) < UNSIGNED(B) then
        result := to_unsigned(1, 32);
      else
        result := (others => '0');
      end if;
    when "011" => -- XOR
      result := UNSIGNED(A xor B);
    when others =>
      result := (others => '0');
  end case;

  -- Assign the final result back to the output port
  ALUResult <= STD_LOGIC_VECTOR(result);
```

soo the explanation will be like this

we give instruction (in this case its instruction memory)

func7 0000000, func3 100, this is going to tell what operations are we going to do, in this case XOR

opcode 0110011, is the opcode for r type instruction. meaning every r type instruction will use this opcode.

rs1, rs2, are given as the question in math. in this case we will do XOR

rd will be the place where we store the result for the question

```

process(op, funct3, funct7)
begin
    case op is
        when "0110011" => -- R-Type instructions (add, sub, and, or) [5, 10]
            RegWrite    <= '1'; -- Result is written to rd [5]
            ALUSrc      <= '0'; -- Use second register rs2 [6]
            Branch      <= '0';
            Jump        <= '0';
            MemWrite    <= '0'; -- No memory write
            ResultSrc   <= "00"; -- Use ALU result

            -- Level 2: ALU Decoder logic based on funct3 and funct7 [2]
            if funct7 = "0000000" then
                if funct3 = "000" then
                    ALUControl <= "010"; -- ADD [7, 8]
                elsif funct3 = "100" then
                    ALUControl <= "011"; -- XOR
                elsif funct3 = "111" then
                    ALUControl <= "000"; -- AND [7, 8]
                elsif funct3 = "110" then
                    ALUControl <= "001"; -- OR [7, 8]
                else
                    ALUControl <= "000"; -- Default (avoid latch)
                end if;
            end if;
        end case;
    end process;
end

```

here is it in my control unit. case op is 0110011, then its r type.

now we set the thing. like the reg write, etc.

and if funct7 is 0000000, then check for funct 3, then tell ALU what to do

okay because i already do the changes, now i will test it compiler explorer

The screenshot shows the Compiler Explorer interface. On the left, the C++ source code for a function named `XOR()` is displayed. The code initializes two integers, `a` and `b`, with values 15 and 10 respectively, and then performs a bitwise XOR operation on them, storing the result in `c`. On the right, the corresponding assembly code generated by the compiler is shown. The assembly code includes instructions for stack frame setup, variable initialization, and the XOR operation itself, which is highlighted in blue. The assembly code ends with a `ebreak` instruction.

okay i forgot what do you say about the compiler explorer, but i try to make XOR operation in c.

yet the compiler explorer give me some things that is random. but the main part is the XOR.

a5 a4 a5. bur if i were me. here will be my code

```

addi x1, x0, 15      # x1 = 15
addi x2, x0, 10      # x2 = 10
xor  x3, x1, x2      # x3 = 5 (15 XOR 10)

```

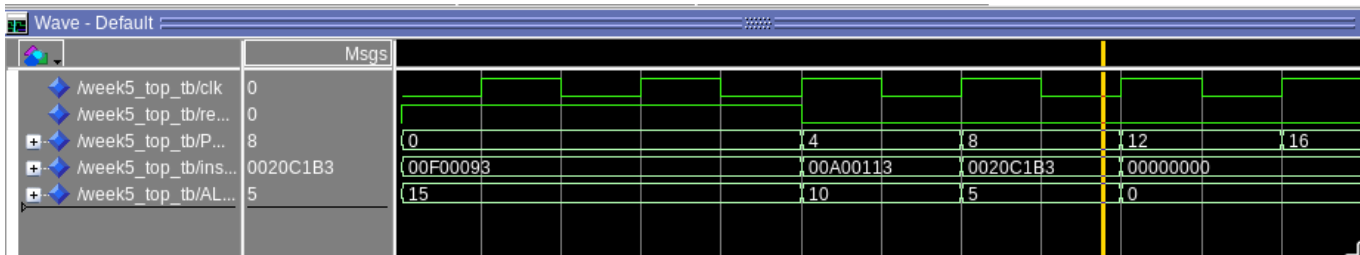
```

architecture behavioral of Instruction_Memory is
  -- ROM array containing test program with XOR and other R-Type instructions
  type rom_type is array (0 to 63) of STD_LOGIC_VECTOR(31 downto 0);
  constant ROM : rom_type := (
    -- Test Program: XOR and R-Type Instructions
    0 => X"00F00093", -- addi x1, x0, 15      (x1 = 15)
    1 => X"00A00113", -- addi x2, x0, 10      (x2 = 10)
    2 => X"0020C1B3", -- xor  x3, x1, x2      (x3 = 15 XOR 10 = 5)

    others => X"00000000" -- nop
  );

```

low and behold, here is the testbench



heyy it works. (after an hour of recalculating the instruction i should use :))(func3 instead of 100 i got 001)

I-Type

Intermediate type, its kinda the same like R type, but here, we actually give a new value rather than we calculate saved value.

here in i type, we didnt use func7 and rs2. we use both of the lanes (func 7 7bits, and rs2 5bits) for an immediate value (12 bits)

here i will change the control unit so it accepts my XORI

```
when "0010011" => -- I-Type arithmetic instructions (addi, xori, etc.)
    RegWrite    <= '1';    -- Save result to register
    ALUSrc      <= '1';    -- Use immediate constant
    Branch      <= '0';
    Jump        <= '0';
    MemWrite    <= '0';    -- No memory write
    ResultSrc   <= "00";   -- Use ALU result

    -- Check funct3 to determine which I-Type operation
    if funct3 = "000" then
        ALUControl <= "010"; -- ADDI (ADD)
    elsif funct3 = "100" then
        ALUControl <= "011"; -- XORI (XOR)
    else
        ALUControl <= "010"; -- Default to ADD
    end if;
```

sooo it gives the ALU 011, but wait. alu doesnt have rs2. how would alu take the value from imm ?

in my top level file, i have this

```
-- 3. EXECUTE STAGE
-- Mux for ALU Source B
SrcB <= ImmExt_out when ALUSrc = '1' else RD2;
```

```
CoreALU: entity work ALU port map(RD1, SrcB, ALUControl
```

so because i already set ALUSrc to 1, it will use my immediate. nice

the rest is the same. the alu will count the XOR, and i will show you the graph again

architecture behavioral of Instruction_Memory is

-- ROM array containing test program

type rom_type is array (0 to 63) of STD_LOGIC_VECTOR(31 downto 0);

constant ROM : rom_type := (

-- Test Program: XOR (R-Type) and XORI (I-Type)

0 => X"00F00093", -- addi x1, x0, 15 (x1 = 15)

1 => X"00A00113", -- addi x2, x0, 10 (x2 = 10)

2 => X"0140C213", -- xori x3, x1, 20 (x3 15 XOR 20 = 27)

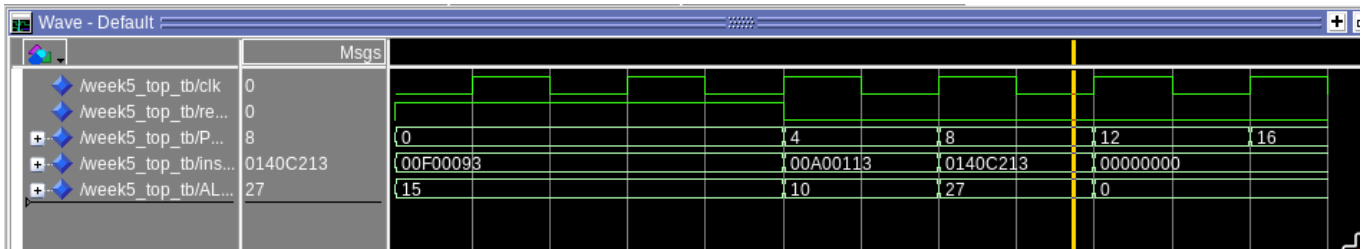
others => X"00000000" -- nop

);

begin

there you go, a test program with instruction to do XOR with x1, and 20, save it to x3

now if i will run this



work flawlesslyyyy

now for the final part

B-Type

im going to do BNE

BNE

before, i already have branch on equal, which makes if rs1 and rs2 is the same, then i can jump with the IMM, which is spitted accross the instruction.



diffrent from I and R type, this one is cursed.

you will have 12, 10: 5 of your IMM, thenin the second IMM, yo can get your 4 : 1 ANDDD YOUR LEFT OUT 1 BIT 11 WHERE YOU MISS IT IN THE FIRST IMM.

so now, i have to split my imm into several parts

imm 12, imm 11, imm 10 - 5, imm 4 - 1. **nice**

now to detect if its BNE or BEQ

```

57  I  -- Branch decision:
58      -- BEQ (funct3=000): Branch when Zero=1 (equal)
59      -- BNE (funct3=001): Branch when Zero=0 (not equal)
60      PCSrc <= (Branch AND Zero) when instr(12) = '0' else      -- BEQ
61      (Branch AND (not Zero));                                  -- BNE

```

i put this in my top file, basically checking the instnction. this is a very lazy check because im just checking the bit 12 of the instruction memory. but its enough.

so its the same logic as BEQ. it checks for zero.

here in BNE, we check if the value is not the same (so he SUB value would not be zero), if yes then we jump.

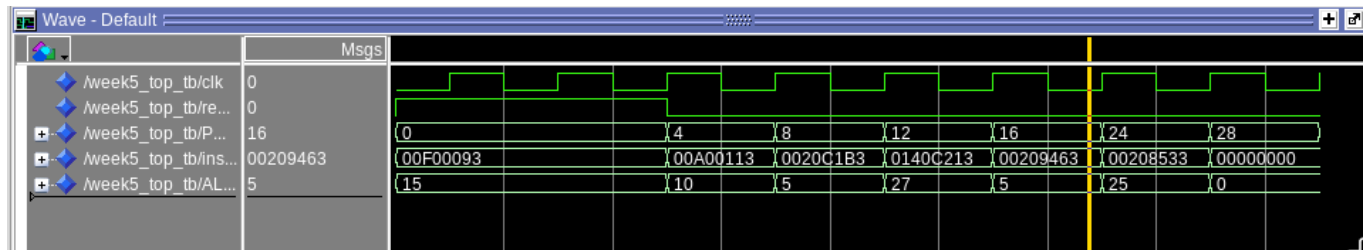
```

12
13 architecture behavioral of Instruction_Memory is
14     -- ROM array containing test program
15     type rom_type is array (0 to 63) of STD_LOGIC_VECTOR(31 downto 0);
16     constant ROM : rom_type := (
17         -- Test Program: XOR (R-Type), XORI (I-Type), BNE (B-Type)
18         0 => X"00F00093", -- addi x1, x0, 15    (x1 = 15)
19         1 => X"00A00113", -- addi x2, x0, 10    (x2 = 10)
20         2 => X"0020C1B3", -- xor  x3, x1, x2    (x3 = 15 XOR 10 = 5)
21         3 => X"0140C213", -- xori x4, x1, 20    (x4 = 15 XOR 20 = 27)
22         4 => X"00209463", -- bne x1, x2, 8    (x1≠x2? Yes, jump to PC+8=20)
23         5 => X"00000013", -- nop (skipped!)
24         6 => X"00208533", -- add x10, x1, x2    (should execute: x10=25)
25
26         others => X"00000000" -- nop
27     );
28 begin
29     -- Word-aligned memory access: use bits [7 downto 2]

```

this is my instruction memory.

lets test this



yayy it jumped. nice

it even show the subtraction result ($15 - 10 = 5$, this is the x1 and x2. so because both of them is not the same, then he jumped. yay)

i hope this is enough. and if its not enough, please let me know. i will be more than happy to pass this course. thankyouu